

Primeros pasos en Python

Pensamiento Computacional

Facultad de Ciencias Exactas y Naturales, UBA

Segundo Cuatrimestre 2026

INICIALES

Organización de la Materia

- **Miércoles:** clase teórico-práctica, en aula.
- **Viernes/Sábado:** clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- **Entregas:** a través del **Campus de Exactas** (<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- **Dos parciales en el laboratorio:**
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- **Las notas de los parciales se promedian:**
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- **Luego, si el promedio es P :**
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas** (<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas** (<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas** (<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas** (<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: rinden un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: **rinden** un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: **rinden** un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: **rinden** un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recursar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas** (<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: **rinden** un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recurrar la materia.

Organización de la Materia

- Miércoles: clase teórico-práctica, en aula.
- Viernes/Sábado: clase en laboratorio. Tomamos lista: hay que tener 80 % de asistencia.
- Entregas: a través del **Campus de Exactas**
(<https://campus.exactas.uba.ar>). Estudiantes del CBC: es **otro** campus distinto al del CBC, les vamos a crear usuario, revisen el mail.
- Dos parciales en el laboratorio:
 - El primero, más conceptual, con poca o nula producción de código.
 - El segundo, escribiendo código propio.
- Las notas de los parciales se promedian:
 - Si los parciales están aprobados (≥ 4), el promedio se redondea hacia arriba.
 - Si alguno de los parciales está desaprobado, se redondea hacia abajo.
- Luego, si el promedio es P :
 - $P \geq 7$: promociona. Felicitaciones.
 - $4 \leq P < 7$: van a final. ¡A remar!
 - $P < 4$ y un parcial aprobado: **rinden** un recuperatorio integrador. La nota del recuperatorio reemplaza la del parcial.
 - Ambos parciales desaprobados: deben recurrar la materia.

¿CÓMO VAMOS A TRABAJAR?

Modelar

- Una de las tareas que se hacen cuando se programa es representar algún aspecto del mundo real, se lo llama **modelar**.

Modelar

- Una de las tareas que se hacen cuando se programa es representar algún aspecto del mundo real, se lo llama **modelar**.
- Un modelo es una **representación**, un recorte de la realidad que incluye los aspectos que nos interesan. . .

Modelar

- Una de las tareas que se hacen cuando se programa es representar algún aspecto del mundo real, se lo llama **modelar**.
- Un modelo es una **representación**, un recorte de la realidad que incluye los aspectos que nos interesan. . . , pero para saber qué nos interesa y qué no, **¡hay que entender muy bien el problema!**

Modelar

- Una de las tareas que se hacen cuando se programa es representar algún aspecto del mundo real, se lo llama **modelar**.
- Un modelo es una **representación**, un recorte de la realidad que incluye los aspectos que nos interesan. . . , pero para saber qué nos interesa y qué no, **¡hay que entender muy bien el problema!**
- La computadora es una herramienta **fundamental** para modelar y nos puede ayudar a entender.

Guía

1 Entender el problema

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:
 - ¿Cómo conviene almacenar la información?

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:
 - ¿Cómo conviene almacenar la información?
 - ¿Cómo la procesamos para producir el resultado?

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:
 - ¿Cómo conviene almacenar la información?
 - ¿Cómo la procesamos para producir el resultado?
- 7 **Programar**, traduciendo la secuencia de pasos del punto 5 a código.

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:
 - ¿Cómo conviene almacenar la información?
 - ¿Cómo la procesamos para producir el resultado?
- 7 **Programar**, traduciendo la secuencia de pasos del punto 5 a código.
- 8 **Evaluar la solución**: ¿tiene sentido? ¿Cómo nos damos cuenta si cometimos algún error? Si no cometimos errores, ¿cómo debemos interpretar la solución?

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:
 - ¿Cómo conviene almacenar la información?
 - ¿Cómo la procesamos para producir el resultado?
- 7 **Programar**, traduciendo la secuencia de pasos del punto 5 a código.
- 8 **Evaluar la solución**: ¿tiene sentido? ¿Cómo nos damos cuenta si cometimos algún error? Si no cometimos errores, ¿cómo debemos interpretar la solución?

1 a 5 no tienen que ver con la compu: es algo que conviene hacer **siempre** que tenemos que resolver un problema complejo.

Guía

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes para responder la pregunta? ¿cuáles se pueden eliminar?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu:
 - ¿Cómo conviene almacenar la información?
 - ¿Cómo la procesamos para producir el resultado?
- 7 **Programar**, traduciendo la secuencia de pasos del punto 5 a código.
- 8 **Evaluar la solución**: ¿tiene sentido? ¿Cómo nos damos cuenta si cometimos algún error? Si no cometimos errores, ¿cómo debemos interpretar la solución?

1 a 5 no tienen que ver con la compu: es algo que conviene hacer **siempre** que tenemos que resolver un problema complejo.

6 requiere *saber* qué se puede hacer en la compu, pero no usarla (todavía).

Organización de la materia

En las clases de los miércoles vamos a:

- Introducir un problema o complejizar un problema conocido.
- Discutir los puntos 1 a 5 del esquema anterior.
- Presentar formas de almacenar los datos y procedimientos computacionales que nos puedan servir para atacar el problema (punto 6).

Organización de la materia

En las clases de los miércoles vamos a:

- Introducir un problema o complejizar un problema conocido.
- Discutir los puntos 1 a 5 del esquema anterior.
- Presentar formas de almacenar los datos y procedimientos computacionales que nos puedan servir para atacar el problema (punto 6).

Organización de la materia

En las clases de los miércoles vamos a:

- Introducir un problema o complejizar un problema conocido.
- Discutir los puntos 1 a 5 del esquema anterior.
- Presentar formas de almacenar los datos y procedimientos computacionales que nos puedan servir para atacar el problema (punto 6).

En los labos vamos a:

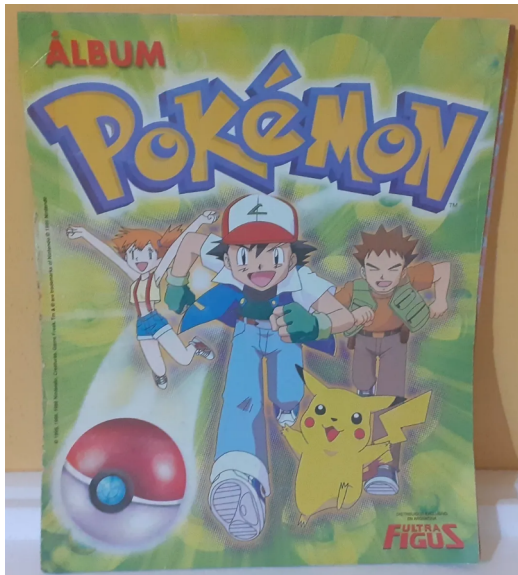
- Hacer un **breve** repaso del problema.
- Recordar y completar la exposición sobre estructuras de datos y procedimientos.
- **¡Programar!**
- Evaluar las soluciones: paso 8.

FIGURITAS

Mundial 2026



No todo es fútbol



¡Vamos las chicas!



El desafío: Álbum Vacío



La satisfacción: Álbum Lleno



El álbum 2026



- 48 selecciones (2022: 32).

El álbum 2026



- 48 selecciones (2022: 32).
- 980 figuritas (2022: 670, contado extras).

El álbum 2026



- 48 selecciones (2022: 32).
- 980 figuritas (2022: 670, contado extras).
- Cada sobre viene con 7 figuritas (2022: 5)

Completando el álbum:

- Álbum con 980 figuritas.
- Cada figurita se imprime en cantidades iguales y se distribuye aleatoriamente.
- Cada paquete incluye siete (7) figuritas.

¿Qué preguntas podemos plantear?



Completando el álbum:

- Álbum con 980 figuritas.
- Cada figurita se imprime en cantidades iguales y se distribuye aleatoriamente.
- Cada paquete incluye siete (7) figuritas.

¿Qué preguntas podemos plantear?



- ¿Cuántos paquetes hay que comprar para llenar el álbum?

Completando el álbum:

- Álbum con 980 figuritas.
- Cada figurita se imprime en cantidades iguales y se distribuye aleatoriamente.
- Cada paquete incluye siete (7) figuritas.

¿Qué preguntas podemos plantear?



- ¿Cuántos paquetes hay que comprar para llenar el álbum?
- **EN PROMEDIO**, ¿cuántos paquetes hay que comprar para llenar el álbum?

Completando el álbum:

- Álbum con 980 figuritas.
- Cada figurita se imprime en cantidades iguales y se distribuye aleatoriamente.
- Cada paquete incluye siete (7) figuritas.

¿Qué preguntas podemos plantear?



- ¿Cuántos paquetes hay que comprar para llenar el álbum?
- **EN PROMEDIO**, ¿cuántos paquetes hay que comprar para llenar el álbum?
- ¿Qué chances tengo de completar el álbum si puedo comprar a lo sumo 800 paquetes?

A JUGAR

Simplificando

¿Cómo podemos experimentar sin comprar figuritas?

Simplificando

¿Cómo podemos experimentar sin comprar figuritas?

Para familiarizarnos con el problema y poder plantearlo adecuadamente, vamos a empezar **jugando** una versión reducida del problema.

Simplificando

¿Cómo podemos experimentar sin comprar figuritas?

Para familiarizarnos con el problema y poder plantearlo adecuadamente, vamos a empezar **jugando** una versión reducida del problema.

Para esto, **necesitamos simplificar un poco el problema.**

Simplificando

¿Cómo podemos experimentar sin comprar figuritas?

Para familiarizarnos con el problema y poder plantearlo adecuadamente, vamos a empezar **jugando** una versión reducida del problema.

Para esto, **necesitamos simplificar un poco el problema**.

Antes de simplificar, pensemos:

- ¿Qué elementos deberíamos modelar?
- ¿Qué elementos del problema podrían eliminarse sin afectar las preguntas que nos interesan?
- ¿Qué elementos del problema podrían simplificarse o reducirse?

Simplificando

¿Cómo podemos experimentar sin comprar figuritas?

Para familiarizarnos con el problema y poder plantearlo adecuadamente, vamos a empezar **jugando** una versión reducida del problema.

Para esto, **necesitamos simplificar un poco el problema**.

Antes de simplificar, pensemos:

- ¿Qué elementos deberíamos modelar?
- ¿Qué elementos del problema podrían eliminarse sin afectar las preguntas que nos interesan?
- ¿Qué elementos del problema podrían simplificarse o reducirse?

La simplificación puede *deformar un poco el problema*. La idea es hacer simplificaciones que no alteren demasiado lo esencial y que, eventualmente, podamos deshacer una vez que hayamos resuelto la versión simplificada.

A pensar

Simplificación: se venden figuritas **individualmente**

- Álbum con **seis** figuritas.
- Cada figurita se imprime en cantidades iguales y se distribuye aleatoriamente.
- Compramos figuritas *de a una*.
- EN PROMEDIO, ¿cuántas figuritas hay que comprar para llenar el álbum?
- ¿Qué chances tengo de completar el álbum si puedo comprar a lo sumo 11 figuritas?

Simplificación: se venden figuritas **individualmente**

- Álbum con **seis** figuritas.
- Cada figurita se imprime en cantidades iguales y se distribuye aleatoriamente.
- Compramos figuritas *de a una*.
- EN PROMEDIO, ¿cuántas figuritas hay que comprar para llenar el álbum?
- ¿Qué chances tengo de completar el álbum si puedo comprar a lo sumo 11 figuritas?

Compremos figuritas y...
¡a experimentar!

Nuestro album de juguete

Vamos a realizar un experimento:

Llenar un álbum de figuritas.

Para comprar una figurita, tiramos un dado y éste indicará cuál figurita salió.
Las vamos pegando hasta llenar el álbum, y registramos cuántas tuvimos que comprar en total.

El álbum						
	0	1	2	3	4	5

Vamos a llenar el álbum.

En la siguiente tabla registramos la cantidad de figuritas que hubo que comprar al llenar el álbum.

Llenado de álbum	
Cantidad comprada	

Primera actividad: la vamos a hacer en grupos de a dos

Materiales, van a recibir:

- Un dado con caras numeradas del 0 al 5.
- Un álbum vacío por grupo.

Primera actividad: la vamos a hacer en grupos de a dos

Materiales, van a recibir:

- Un dado con caras numeradas del 0 al 5.
- Un álbum vacío por grupo.

¿Qué hacemos?:

- Vamos a comprar figuritas de **a una** por vez.

Primera actividad: la vamos a hacer en grupos de a dos

Materiales, van a recibir:

- Un dado con caras numeradas del 0 al 5.
- Un álbum vacío por grupo.

¿Qué hacemos?:

- Vamos a comprar figuritas de **a una** por vez.
- Al tirar el dado, **compramos** una *figu*.

Primera actividad: la vamos a hacer en grupos de a dos

Materiales, van a recibir:

- Un dado con caras numeradas del 0 al 5.
- Un álbum vacío por grupo.

¿Qué hacemos?:

- Vamos a comprar figuritas de **a una** por vez.
- Al tirar el dado, **compramos** una *figu*.
- **Registramos cuántas** figs tuvimos que comprar hasta completar el álbum.

Primera actividad: la vamos a hacer en grupos de a dos

Materiales, van a recibir:

- Un dado con caras numeradas del 0 al 5.
- Un álbum vacío por grupo.

¿Qué hacemos?:

- Vamos a comprar figuritas de **a una** por vez.
- Al tirar el dado, **compramos** una *figu*.
- **Registramos cuántas** figus tuvimos que comprar hasta completar el álbum.
- ¡Premio al que completó antes!

Primera actividad: la vamos a hacer en grupos de a dos

Materiales, van a recibir:

- Un dado con caras numeradas del 0 al 5.
- Un álbum vacío por grupo.

¿Qué hacemos?:

- Vamos a comprar figuritas de **a una** por vez.
- Al tirar el dado, **compramos** una *figu*.
- **Registramos cuántas** figus tuvimos que comprar hasta completar el álbum.
- ¡Premio al que completó antes!

Puesta en común. Hacemos promedio entre todos, lo visualizamos.

PARA ESTO NO QUEDA OTRA QUE **¡PROGRAMAR!**

Lenguaje de programación

- Una computadora necesita que le digamos qué hacer, le deberemos dar **instrucciones**.

Lenguaje de programación

- Una computadora necesita que le digamos qué hacer, le deberemos dar **instrucciones**.
- Un **lenguaje de programación** es, esencialmente, una forma de dar instrucciones a la computadora. Además, resulta comprensible para un humano.

Lenguaje de programación

- Una computadora necesita que le digamos qué hacer, le deberemos dar **instrucciones**.
- Un **lenguaje de programación** es, esencialmente, una forma de dar instrucciones a la computadora. Además, resulta comprensible para un humano.
- Hay **muchos** lenguajes de programación.

Lenguaje de programación

- Una computadora necesita que le digamos qué hacer, le deberemos dar **instrucciones**.
- Un **lenguaje de programación** es, esencialmente, una forma de dar instrucciones a la computadora. Además, resulta comprensible para un humano.
- Hay **muchos** lenguajes de programación.
- Nosotros vamos a programar en **Python**.

Lenguaje de programación

- Una computadora necesita que le digamos qué hacer, le deberemos dar **instrucciones**.
- Un **lenguaje de programación** es, esencialmente, una forma de dar instrucciones a la computadora. Además, resulta comprensible para un humano.
- Hay **muchos** lenguajes de programación.
- Nosotros vamos a programar en **Python**.
- Es el lenguaje **más usado** desde hace varios años.

Lenguaje de programación

- Una computadora necesita que le digamos qué hacer, le deberemos dar **instrucciones**.
- Un **lenguaje de programación** es, esencialmente, una forma de dar instrucciones a la computadora. Además, resulta comprensible para un humano.
- Hay **muchos** lenguajes de programación.
- Nosotros vamos a programar en **Python**.
- Es el lenguaje **más usado** desde hace varios años.
- `Python` está diseñado para que el programa resulte fácil de leer y entender por un humano.

¿Dónde programamos?

- Vamos a trabajar con **Spyder**.

¿Dónde programamos?

- Vamos a trabajar con **Spyder**.
- `Spyder` es un **entorno de desarrollo**, que nos permite escribir y ejecutar código escrito en `Python` y nos da algunas otras herramientas.

¿Dónde programamos?

- Vamos a trabajar con **Spyder**.
- `Spyder` es un **entorno de desarrollo**, que nos permite escribir y ejecutar código escrito en `Python` y nos da algunas otras herramientas.
- Está instalado en las computadoras de los laboratorios que vamos a usar.

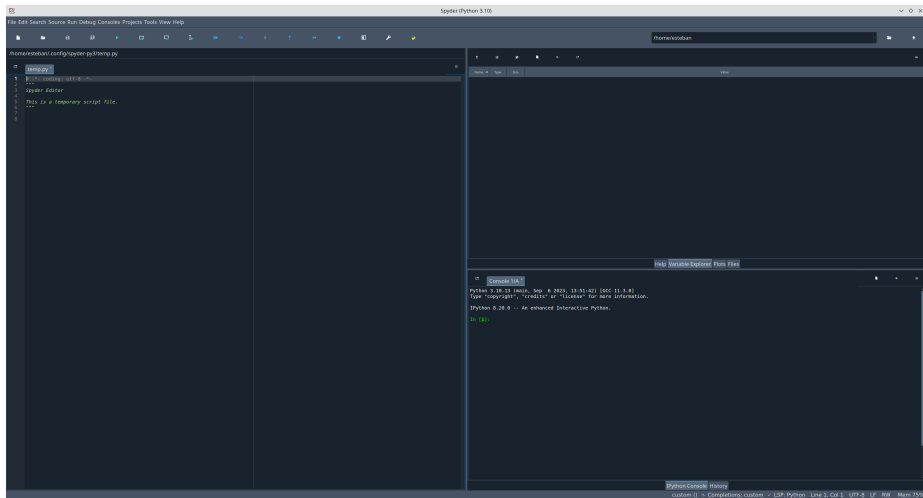
¿Dónde programamos?

- Vamos a trabajar con **Spyder**.
- `Spyder` es un **entorno de desarrollo**, que nos permite escribir y ejecutar código escrito en `Python` y nos da algunas otras herramientas.
- Está instalado en las computadoras de los laboratorios que vamos a usar.
- Es **gratis** y lo pueden instalar en sus computadoras (funciona en Windows, Linux y Mac).

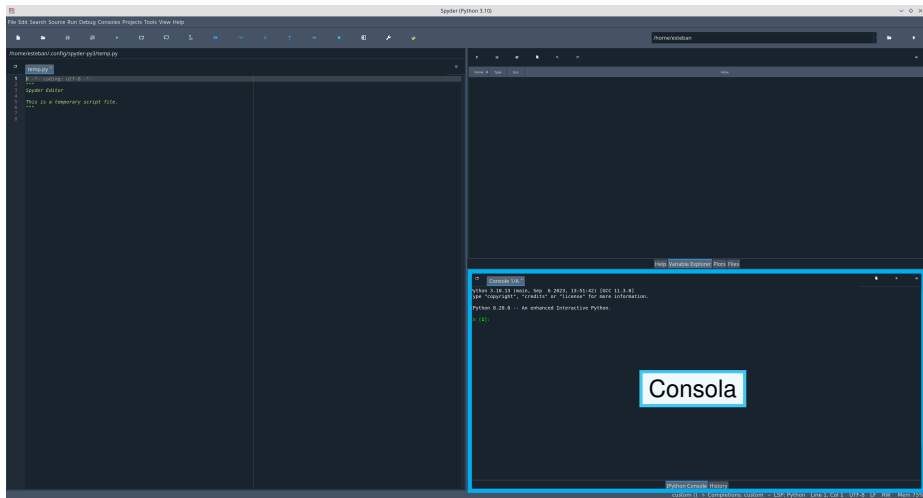
¿Dónde programamos?

- Vamos a trabajar con **Spyder**.
- `Spyder` es un **entorno de desarrollo**, que nos permite escribir y ejecutar código escrito en `Python` y nos da algunas otras herramientas.
- Está instalado en las computadoras de los laboratorios que vamos a usar.
- Es **gratis** y lo pueden instalar en sus computadoras (funciona en Windows, Linux y Mac).
- Veamos cómo funciona. . .

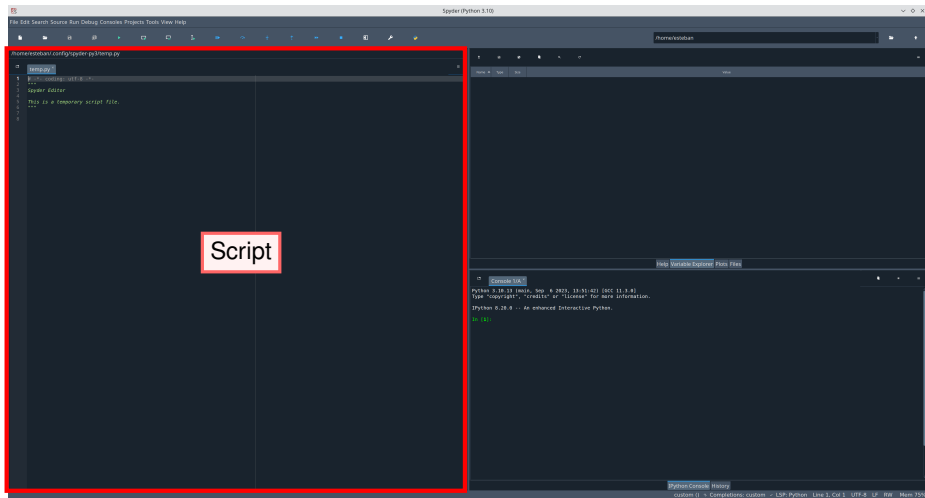
¿Cómo se ve el Spyder?



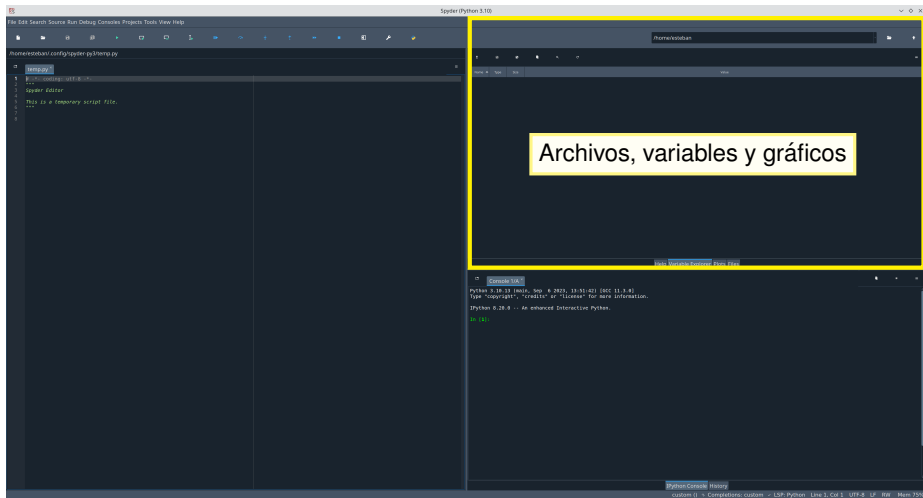
¿Cómo se ve el Spyder?



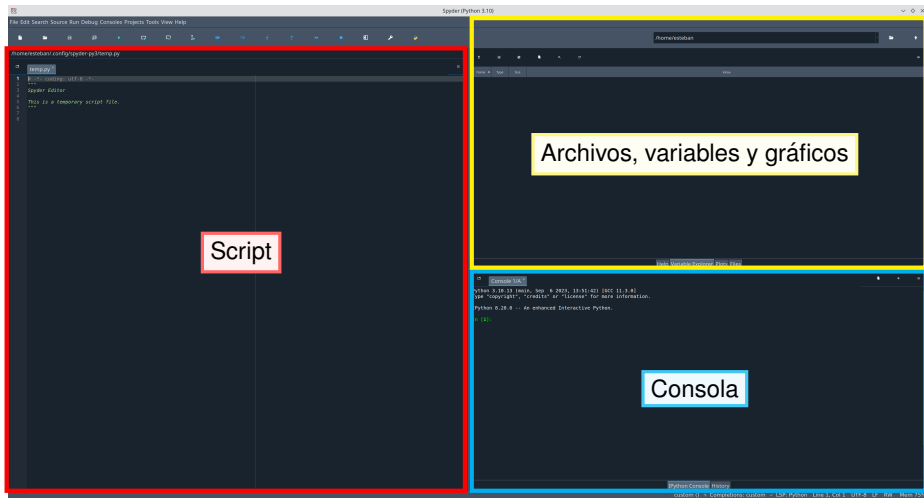
¿Cómo se ve el Spyder?



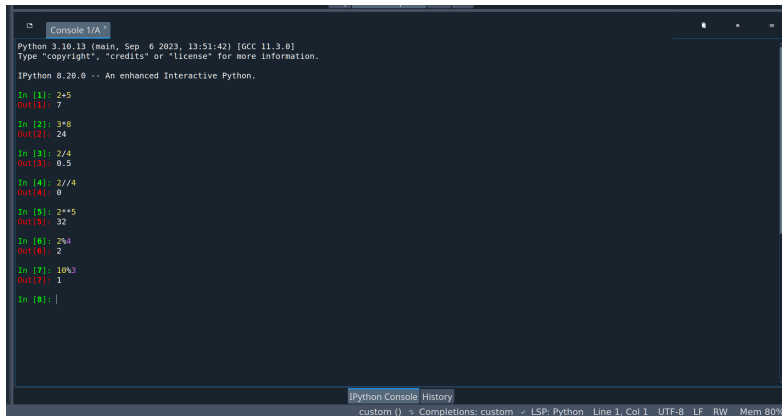
¿Cómo se ve el Spyder?



¿Cómo se ve el Spyder?



La consola me permite introducir expresiones escritas en Python y evaluarlas



The screenshot shows a terminal window titled "Console 1/A *". It displays the output of running Python 3.10.13, including version information and the IPython 8.20.0 prompt. Below the prompt, several arithmetic expressions are entered and evaluated:

```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

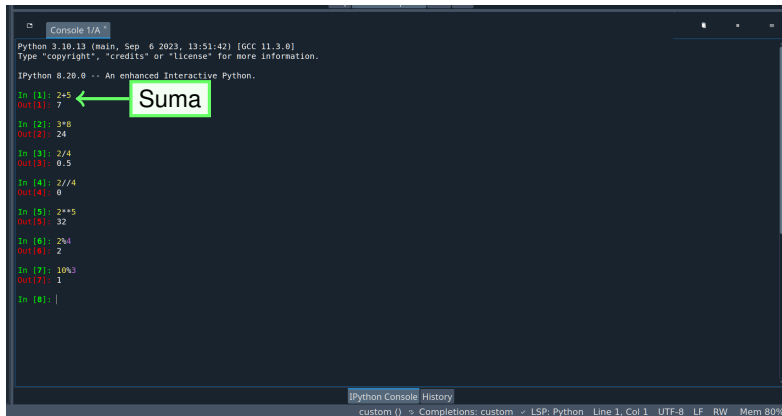
In [8]: |
```

The bottom status bar of the window shows "Python Console History", "custom ()", "Completions: custom", "LSP: Python", "Line 1, Col 1", "UTF-8", "LF", "RW", and "Mem 80%".

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas



```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

In [8]: |
```

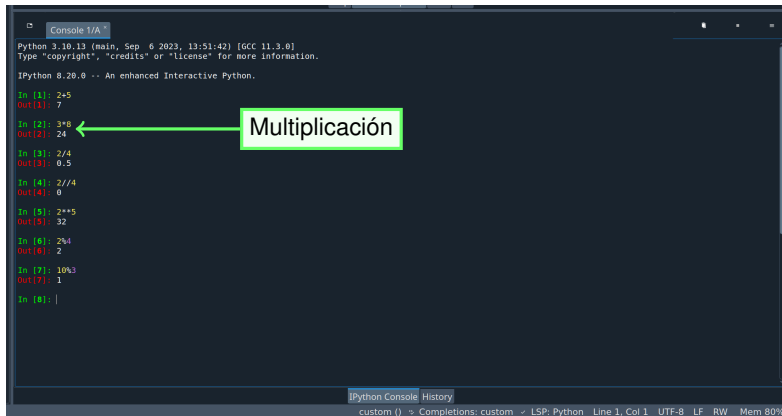
Python Console History

custom () Completions: custom ✓ LSP: Python Line 1, Col 1 UTF-8 LF RW Mem 80%

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas



```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

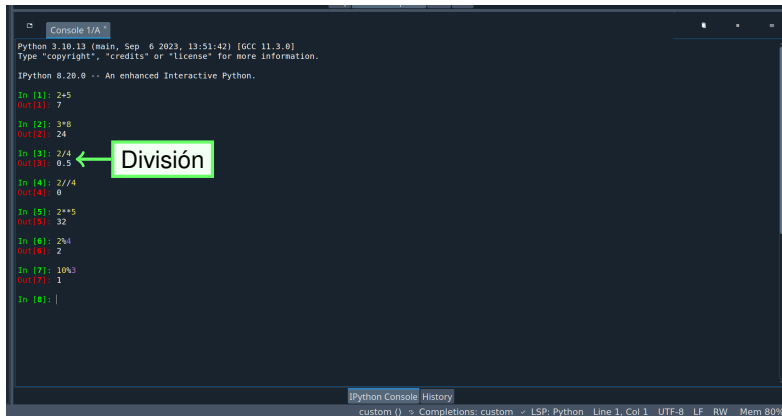
In [8]: |
```

The screenshot shows a Python console window with a dark theme. The title bar reads 'Console 1/A *'. The text inside shows the Python version (3.10.13) and IPython version (8.20.0). A series of arithmetic operations are entered and evaluated. A green box with the text 'Multiplicación' is positioned over the third input line, 'In [2]: 3*8', with a green arrow pointing to it. The output for this operation is 'Out[2]: 24'. The bottom status bar shows 'Python Console History', 'custom ()', 'Completions: custom', 'LSP: Python', 'Line 1, Col 1', 'UTF-8', 'LF', 'RW', and 'Mem 80%'.

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas



```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4 ← División
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

In [8]: |
```

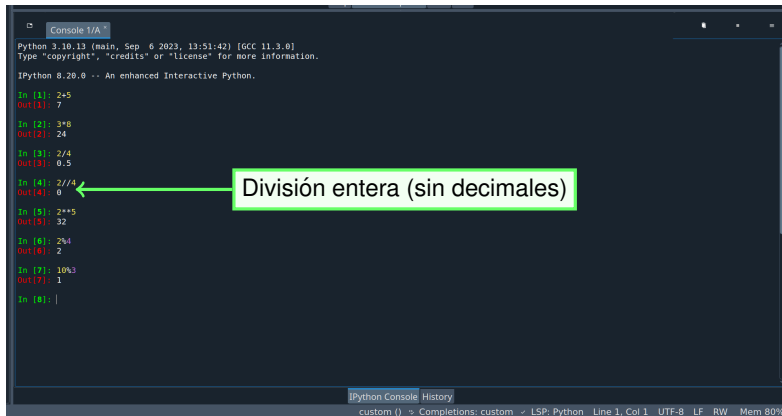
Python Console History

custom () ▾ Completions: custom ▾ LSP: Python Line 1, Col 1 UTF-8 LF RW Mem 80%

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas



```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

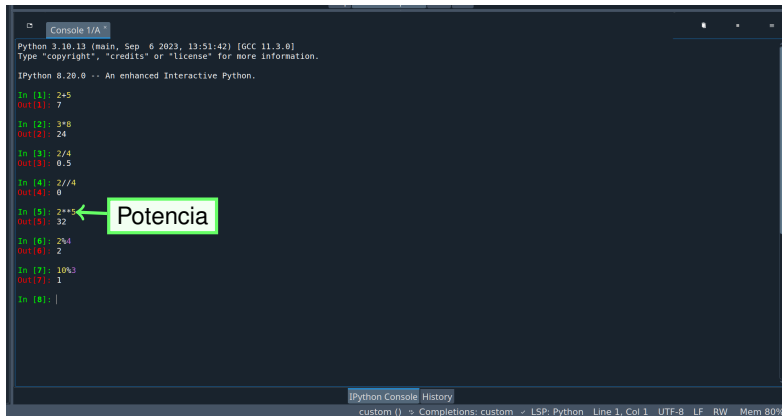
In [8]: |
```

División entera (sin decimales)

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas



```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

In [8]: |
```

The screenshot shows a Python console window with a dark background. The title bar says 'Console 1/A *'. The text inside shows the Python version (3.10.13) and IPython version (8.20.0). It lists several arithmetic operations and their results. A green box with the word 'Potencia' (Power) and an arrow points to the input '2**5' in the 5th line of input, which resulted in '32'.

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas

```

Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

In [8]: |
  
```

Resto de la división entera

Python Console History

custom () Completions: custom ✓ LSP: Python Line 1, Col 1 UTF-8 LF RW Mem 80%

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola me permite introducir expresiones escritas en Python y evaluarlas

The screenshot shows a Python console window with the following content:

```

Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [1]: 2+5
Out[1]: 7

In [2]: 3*8
Out[2]: 24

In [3]: 2/4
Out[3]: 0.5

In [4]: 2//4
Out[4]: 0

In [5]: 2**5
Out[5]: 32

In [6]: 2%4
Out[6]: 2

In [7]: 10%3
Out[7]: 1

In [8]: |
  
```

Green callout boxes with arrows point to specific lines of code:

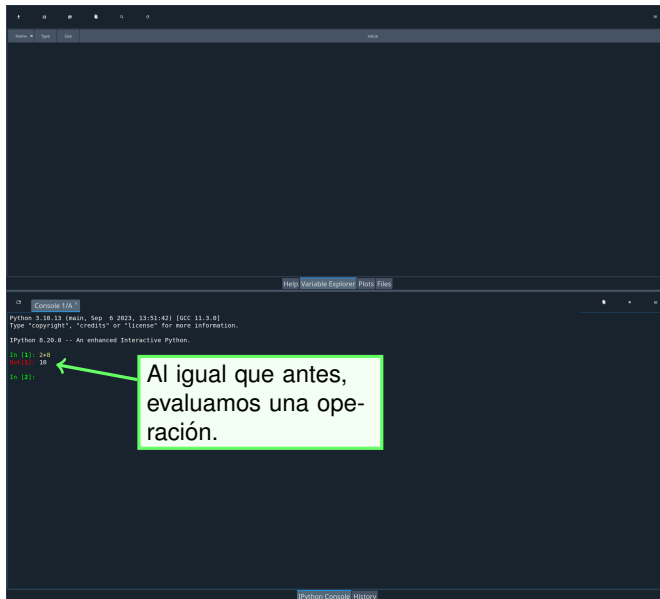
- Suma** points to `In [1]: 2+5`
- Multiplicación** points to `In [2]: 3*8`
- División** points to `In [3]: 2/4`
- División entera (sin decimales)** points to `In [4]: 2//4`
- Potencia** points to `In [5]: 2**5`
- Resto de la división entera** points to `In [6]: 2%4` and `In [7]: 10%3`

The console window title is "Console 1/A". The status bar at the bottom shows "Python Console", "History", and various settings like "LSP: Python", "Line 1, Col 1", "UTF-8", "LF", "RW", and "Mem 80%".

La consola

Se pueden evaluar expresiones (acá la usamos como calculadora). Al apretar la flecha hacia arriba se traen expresiones anteriores (nos ahorramos tipearlas de nuevo).

La consola también me permite guardar los resultados



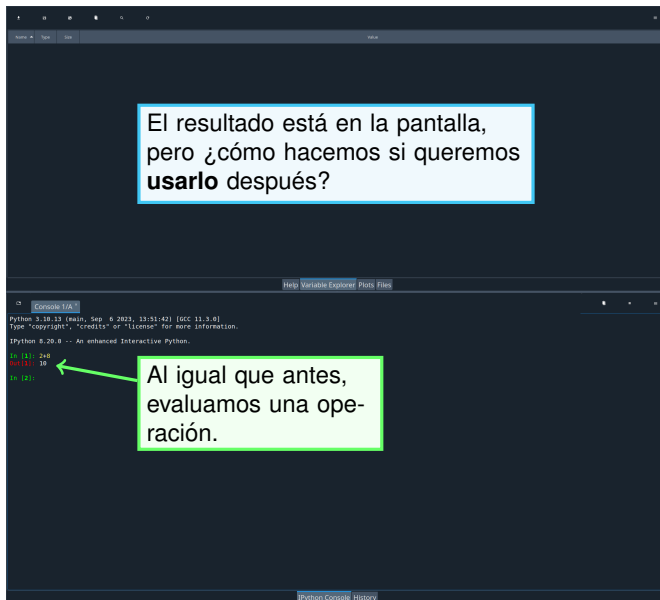
```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

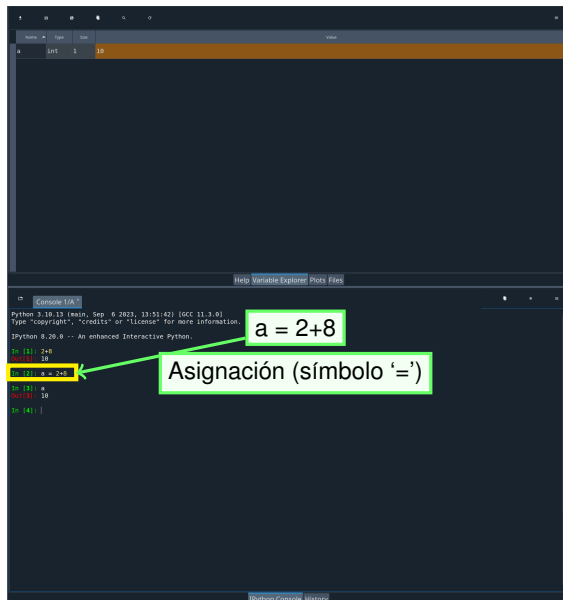
In [1]: 2+0
Out[1]: 10
In [2]:
```

Al igual que antes,
evaluamos una operación.

La consola también me permite guardar los resultados



La consola también me permite guardarme los resultados



The screenshot shows a Jupyter Notebook environment. At the top, a variable explorer table lists a variable 'a' of type 'int' with a value of 10. Below this, the console output shows the execution of Python code. The code includes a variable assignment `a = 2+8`, which is highlighted with a yellow box. A green arrow points from a text box labeled 'Asignación (símbolo '=')' to this line of code. The console also shows the output of `2+8` as 10, and the output of `a` as 10.

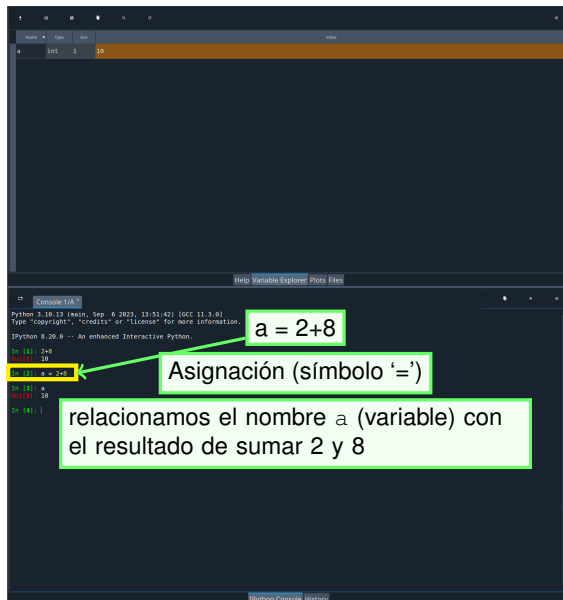
Name	Type	Size	Value
a	int	1	10

```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) (GCC 11.3.0)
Type "copyright", "credits" or "license()" for more information.
IPython 8.29.0 -- An enhanced Interactive Python.
>>> 2+8
10
>>> a = 2+8
>>> a
10
>>>
```

a = 2+8

Asignación (símbolo '=')

La consola también me permite guardarme los resultados



The screenshot shows a Jupyter Notebook interface. At the top, there is a table with columns 'Name', 'Type', and 'Value'. The table contains one row: 'a', 'int', and '10'. Below the table, the main area of the notebook is empty. At the bottom, there is a 'Console' tab. The console shows the following text: 'Python 3.10.13 (main, Sep. 6 2023, 13:51:42) (GCC 11.3.0)' and 'Type "copyright", "credits" or "license()" for more information.' Below this, it says 'IPython 8.20.0 -- An enhanced Interactive Python.' Then, there are three lines of code: 'In [11]: 2+8', 'Out[11]: 10', and 'In [12]: a = 2+8'. The line 'In [12]: a = 2+8' is highlighted with a yellow box. A green arrow points from a text box to this line. The text box contains the text 'Asignación (símbolo \'=\')' and 'relacionamos el nombre a (variable) con el resultado de sumar 2 y 8'. Another text box contains the text 'a = 2+8'.

Name	Type	Value
a	int	10

```
Python 3.10.13 (main, Sep. 6 2023, 13:51:42) (GCC 11.3.0)
Type "copyright", "credits" or "license()" for more information.
IPython 8.20.0 -- An enhanced Interactive Python.

In [11]: 2+8
Out[11]: 10

In [12]: a = 2+8
Out[12]: 10

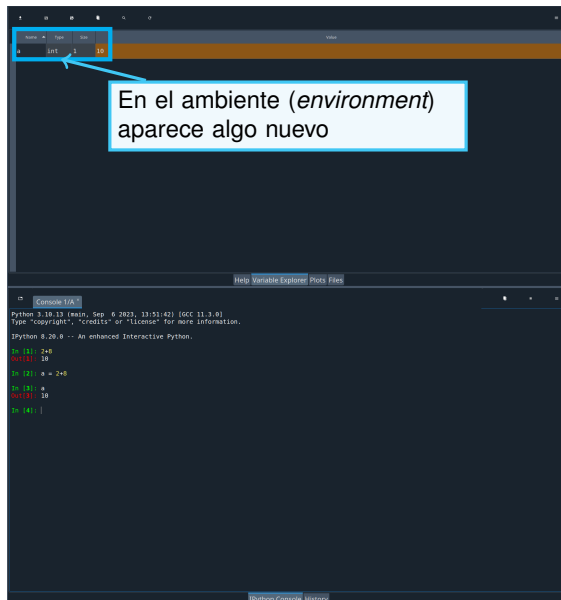
In [13]:
Out[13]:
```

a = 2+8

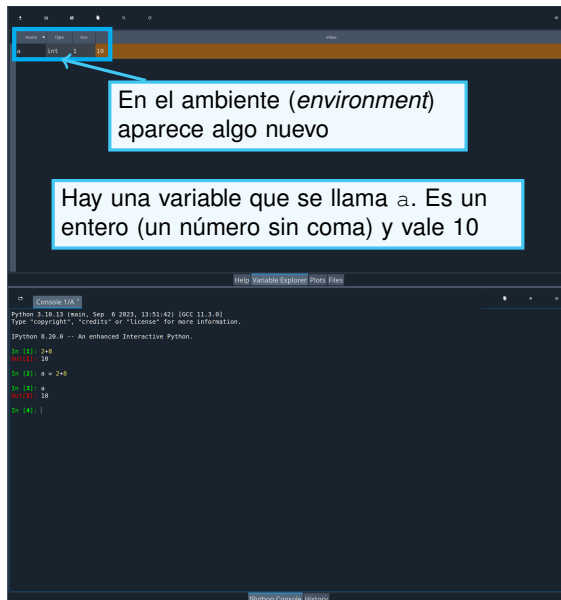
Asignación (símbolo '=')

relacionamos el nombre a (variable) con el resultado de sumar 2 y 8

La consola también me permite guardarme los resultados



La consola también me permite guardarme los resultados



The screenshot shows a Jupyter Notebook interface. At the top, a table in the Variable Explorer displays the variable 'a' with type 'int' and value '10'. A blue box highlights this table, and a blue arrow points from it to a text box. Below the table, another text box explains the variable. At the bottom, the IPython console shows the execution of code that defines 'a' and prints its value.

En el ambiente (*environment*) aparece algo nuevo

Hay una variable que se llama `a`. Es un entero (un número sin coma) y vale 10

```
Python 3.10.13 (main, Sep. 6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.29.0 -- An enhanced Interactive Python.

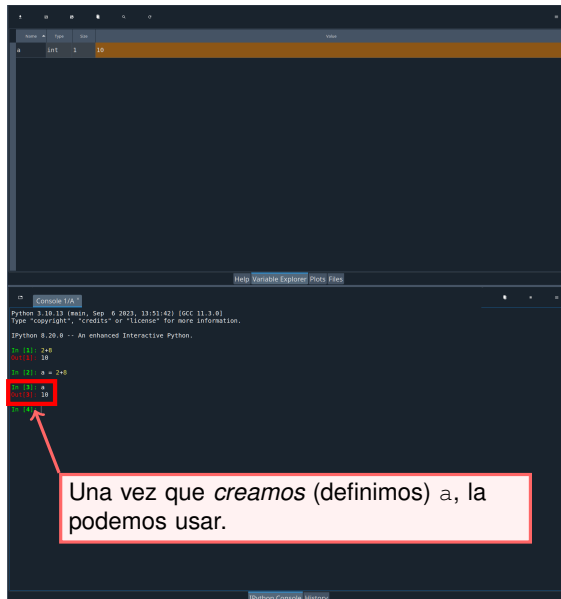
In [1]: 2+8
Out[1]: 10

In [2]: a = 2+8

In [3]: a
Out[3]: 10

In [4]: |
```

La consola también me permite guardarme los resultados



The screenshot shows a Jupyter Notebook interface. At the top, there is a table with columns 'Name', 'Type', and 'Value'. The table contains one row: 'a', 'int', and '10'. Below the table, there is a console window titled 'Console 1/A'. The console shows the following output:

```
Python 3.10.13 (main, Sep  6 2023, 13:51:42) [GCC 11.3.0]
Type "copyright", "credits" or "license()" for more information.

IPython 8.20.0 -- An enhanced Interactive Python.

In [11]: 2+8
Out[11]: 10

In [12]: a = 2+8

In [13]: a
Out[13]: 10

In [14]:
```

An arrow points from the text box below to the output of the last command in the console.

Una vez que *creamos* (definimos) *a*, la podemos usar.

La consola también me permite guardarme los resultados

The screenshot shows a Jupyter Notebook interface. At the top, the 'Variable Explorer' panel displays a table with columns 'Name', 'Type', and 'Value'. It contains one entry: 'a' of type 'int' with a value of '10'. A blue box highlights this entry, with a blue arrow pointing to it from the text 'En el ambiente (*environment*) aparece algo nuevo'.

Below the variable explorer, the 'Console' tab shows the following code and output:

```
Python 3.10.13 (main, Sep. 6 2023, 13:51:42) (GCC 11.3.0)
Type "copyright", "credits" or "license()" for more information.

IPython 8.29.0 -- An enhanced Interactive Python.
--> 11: 2+8
--> 12: a = 2+8
--> 13: a
--> 14: 10
```

A green box highlights the line `a = 2+8`, with a green arrow pointing to it from the text 'Asignación (símbolo '=')'. Another green box highlights the line `a`, with a green arrow pointing to it from the text 'relacionamos el nombre *a* (variable) con el resultado de sumar 2 y 8'. A red box highlights the output `10`, with a red arrow pointing to it from the text 'Una vez que *creamos* (definimos) *a*, la podemos usar.'

¿Qué datos necesitamos manejar en el problema del álbum?

- El álbum (¿cómo representamos un álbum?)

¿Qué datos necesitamos manejar en el problema del álbum?

- El álbum (¿cómo representamos un álbum?)
- Cada una de las figs que sacamos (¿cómo representamos una figu?)

NOMBRES A VALORES: ASIGNACIÓN

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Podemos pensar que la computadora:

- Reservó un **espacio en la memoria**.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Podemos pensar que la computadora:

- Reservó un **espacio en la memoria**.
- Le puso la etiqueta `figu` que nos permite acceder a ese espacio.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Podemos pensar que la computadora:

- Reservó un **espacio en la memoria**.
- Le puso la etiqueta `figu` que nos permite acceder a ese espacio.
- Guardó allí el número 5.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Podemos pensar que la computadora:

- Reservó un **espacio en la memoria**.
- Le puso la etiqueta `figu` que nos permite acceder a ese espacio.
- Guardó allí el número 5.

Asignación

Una figurita se puede representar sencillamente como un número: el número que indica en qué casillero debe pegarse.

La sentencia:

```
figu = 5
```

es una **asignación**. Esta sentencia:

- Genera una **variable**, cuyo nombre es `figu`.
- Le **asigna** a esa variable el valor 5 (un número entero).

Podemos pensar que la computadora:

- Reservó un **espacio en la memoria**.
- Le puso la etiqueta `figu` que nos permite acceder a ese espacio.
- Guardó allí el número 5.

Así, podés **utilizar** el valor almacenado tantas veces como sea necesario.

¿Y el álbum?

¿Cómo hacemos para representar un álbum?

¿Y el álbum?

¿Cómo hacemos para representar un álbum?

Listas

Así como asignamos **un solo número** a una variable, podemos asignarle **una lista** que contenga los valores que uno quiera.

¿Y el álbum?

¿Cómo hacemos para representar un álbum?

Listas

Así como asignamos **un solo número** a una variable, podemos asignarle **una lista** que contenga los valores que uno quiera.

Un ejemplo: `pares = [0, 2, 4, 6, 8, 10]`

De esta manera, estamos **asignando** una lista de valores (0, 2, 4, 6, 8 y 10) a un espacio de memoria con la etiqueta: `pares`.

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.
- Necesitamos modelar un álbum **vacío**.

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.
- Necesitamos modelar un álbum **vacío**.
- Si queremos reservar espacio en la memoria para guardar, digamos, 980 números, necesito 980 *casilleros* y esos casilleros tendrán **algo** adentro (queramos o no).

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.
- Necesitamos modelar un álbum **vacío**.
- Si queremos reservar espacio en la memoria para guardar, digamos, 980 números, necesito 980 *casilleros* y esos casilleros tendrán **algo** adentro (queramos o no).
- Necesitamos **modelar** el álbum de algún modo que sea representable en la computadora.

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.
- Necesitamos modelar un álbum **vacío**.
- Si queremos reservar espacio en la memoria para guardar, digamos, 980 números, necesito 980 *casilleros* y esos casilleros tendrán **algo** adentro (queramos o no).
- Necesitamos **modelar** el álbum de algún modo que sea representable en la computadora.
- Podemos crear una lista llena de **ceros**, de modo que:

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.
- Necesitamos modelar un álbum **vacío**.
- Si queremos reservar espacio en la memoria para guardar, digamos, 980 números, necesito 980 *casilleros* y esos casilleros tendrán **algo** adentro (queramos o no).
- Necesitamos **modelar** el álbum de algún modo que sea representable en la computadora.
- Podemos crear una lista llena de **ceros**, de modo que:
 - Un 0 indica que el casillero del álbum está vacío (sin figu)

Definamos un álbum

- Un álbum recién comprado no contiene ninguna figurita, pero sí los lugares donde pegar las que vayamos consiguiendo.
- Necesitamos modelar un álbum **vacío**.
- Si queremos reservar espacio en la memoria para guardar, digamos, 980 números, necesito 980 *casilleros* y esos casilleros tendrán **algo** adentro (queramos o no).
- Necesitamos **modelar** el álbum de algún modo que sea representable en la computadora.
- Podemos crear una lista llena de **ceros**, de modo que:
 - Un 0 indica que el casillero del álbum está vacío (sin figu)
 - Un 1 indica que el casillero del álbum tiene la figu.

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (`Python` cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (`Python` cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.
- **Modificación:** La misma sintaxis permite modificar casilleros específicos:

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.
- **Modificación:** La misma sintaxis permite modificar casilleros específicos:
 - `album[3] = 1` modifica el **cuarto** casillero de la lista, reemplazándolo por un 1.

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.
- **Modificación:** La misma sintaxis permite modificar casilleros específicos:
 - `album[3] = 1` modifica el **cuarto** casillero de la lista, reemplazándolo por un 1.

```
figu = 4
album[figu] = 1 #Crea una figurita y la pega.
```

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.
- **Modificación:** La misma sintaxis permite modificar casilleros específicos:
 - `album[3] = 1` modifica el **cuarto** casillero de la lista, reemplazándolo por un 1.

```
figu = 4
album[figu] = 1 #Crea una figurita y la pega.
```

- **Sumar:** `sum(album)` suma todos los elementos de la lista.

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.
- **Modificación:** La misma sintaxis permite modificar casilleros específicos:
 - `album[3] = 1` modifica el **cuarto** casillero de la lista, reemplazándolo por un 1.

```
figu = 4
album[figu] = 1 #Crea una figurita y la pega.
```

- **Sumar:** `sum(album)` suma todos los elementos de la lista.
- **Creación rápida:** `[0]*6` crea una lista de ceros de longitud 6. En general, multiplicar una lista por un número produce una nueva lista que repite la lista original tantas veces como diga el número.

Listas - Definiendo un álbum vacío

Para nuestro álbum de juguete podríamos hacer:

```
album = [0,0,0,0,0,0]
```

Que **define** la variable `album` y le asigna una lista de seis (6) ceros.

Las listas admiten distintas operaciones. Por ejemplo:

- **Longitud:** `len(album)` devuelve la longitud de la lista (6).
- **Indexado:** Con corchetes se puede acceder a cada casillero:
 - `album[0]` devuelve el **primer** casillero de lista (Python cuenta desde 0)
 - `album[5]` devuelve el **sexto** casillero de la lista.
 - `album[6]`: en nuestro ejemplo da **error**, porque nos pasamos de la longitud de la lista.
- **Modificación:** La misma sintaxis permite modificar casilleros específicos:
 - `album[3] = 1` modifica el **cuarto** casillero de la lista, reemplazándolo por un 1.

```
figu = 4
album[figu] = 1 #Crea una figurita y la pega.
```

- **Sumar:** `sum(album)` suma todos los elementos de la lista.
- **Creación rápida:** `[0]*6` crea una lista de ceros de longitud 6. En general, multiplicar una lista por un número produce una nueva lista que repite la lista original tantas veces como diga el número.
- Hay **muchas otras** operaciones que se pueden aplicar a listas.

Procedimiento

- Sabemos cómo **representar** las cosas que nuestro programa debe manipular.
- Pero ¿cómo hacemos realmente el proceso de **simular** el llenado de un álbum?

Procedimiento

- Sabemos cómo **representar** las cosas que nuestro programa debe manipular.
- Pero ¿cómo hacemos realmente el proceso de **simular** el llenado de un álbum?
- Hay tres grandes preguntas:
 - ❶ ¿Cómo simulamos la **aleatoriedad** al sacar la figu? (nuestro dado).

Procedimiento

- Sabemos cómo **representar** las cosas que nuestro programa debe manipular.
- Pero ¿cómo hacemos realmente el proceso de **simular** el llenado de un álbum?
- Hay tres grandes preguntas:
 - 1 ¿Cómo simulamos la **aleatoriedad** al sacar la figu? (nuestro dado).
 - 2 ¿Cómo repetimos el proceso de *comprar* la figu y **pegarla** en el álbum.

Procedimiento

- Sabemos cómo **representar** las cosas que nuestro programa debe manipular.
- Pero ¿cómo hacemos realmente el proceso de **simular** el llenado de un álbum?
- Hay tres grandes preguntas:
 - 1 ¿Cómo simulamos la **aleatoriedad** al sacar la figu? (nuestro dado).
 - 2 ¿Cómo repetimos el proceso de *comprar* la figu y **pegarla** en el álbum.
 - 3 ¿Cómo nos damos cuenta cuándo **terminamos** de llenar el álbum?

ALEATORIEDAD

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.
- La razón es sencilla: hay muchísimas y tener todo disponible siempre puede afectar el proceso de desarrollo de código (iremos viendo algunas cosas durante el curso).

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.
- La razón es sencilla: hay muchísimas y tener todo disponible siempre puede afectar el proceso de desarrollo de código (iremos viendo algunas cosas durante el curso).
- El módulo **random** contiene funciones relacionadas con aleatoriedad. Para importarlo escribir al principio del código (arriba de todo):

```
import random
```

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.
- La razón es sencilla: hay muchísimas y tener todo disponible siempre puede afectar el proceso de desarrollo de código (iremos viendo algunas cosas durante el curso).
- El módulo **random** contiene funciones relacionadas con aleatoriedad. Para importarlo escribir al principio del código (arriba de todo):

```
import random
```
- Algunas funciones:
 - `random.random()` : devuelve un número aleatorio entre 0 y 1 (sin incluir 1).

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.
- La razón es sencilla: hay muchísimas y tener todo disponible siempre puede afectar el proceso de desarrollo de código (iremos viendo algunas cosas durante el curso).
- El módulo **random** contiene funciones relacionadas con aleatoriedad. Para importarlo escribir al principio del código (arriba de todo):

```
import random
```

- Algunas funciones:
 - `random.random()`: devuelve un número aleatorio entre 0 y 1 (sin incluir 1).
 - `random.randint(a,b)`: devuelve un número **entero** aleatorio entre *a* y *b* (inclusive), **incluyendo ambos extremos**.

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.
- La razón es sencilla: hay muchísimas y tener todo disponible siempre puede afectar el proceso de desarrollo de código (iremos viendo algunas cosas durante el curso).
- El módulo **random** contiene funciones relacionadas con aleatoriedad. Para importarlo escribir al principio del código (arriba de todo):

```
import random
```

- Algunas funciones:
 - `random.random()`: devuelve un número aleatorio entre 0 y 1 (sin incluir 1).
 - `random.randint(a, b)`: devuelve un número **entero** aleatorio entre *a* y *b* (inclusive), **incluyendo ambos extremos**.
 - `random.shuffle(lista)`: toma una lista y mezcla sus elementos.

Aleatoriedad

- Python no deja disponible directamente todos los posibles recursos que uno puede necesitar.
- Hay funcionalidades específicas que se guardan en *módulos*, que sólo se usan cuando es necesario.
- La razón es sencilla: hay muchísimas y tener todo disponible siempre puede afectar el proceso de desarrollo de código (iremos viendo algunas cosas durante el curso).
- El módulo **random** contiene funciones relacionadas con aleatoriedad. Para importarlo escribir al principio del código (arriba de todo):

```
import random
```

- Algunas funciones:
 - `random.random()`: devuelve un número aleatorio entre 0 y 1 (sin incluir 1).
 - `random.randint(a,b)`: devuelve un número **entero** aleatorio entre *a* y *b* (inclusive), **incluyendo ambos extremos**.
 - `random.shuffle(lista)`: toma una lista y mezcla sus elementos.
 - Hay muchas otras...

En nuestro caso, nos alcanza con la función `random.randint(a,b)`, que nos permite generar un número al azar entre `a` y `b`. Por ejemplo:

En nuestro caso, nos alcanza con la función `random.randint(a,b)`, que nos permite generar un número al azar entre `a` y `b`. Por ejemplo:

```
import random

album = [0,0,0,0,0,0]
figu = random.randint(0,5)
album[figu] = 1
```

En nuestro caso, nos alcanza con la función `random.randint(a,b)`, que nos permite generar un número al azar entre `a` y `b`. Por ejemplo:

```
import random

album = [0,0,0,0,0,0]
figu = random.randint(0,5)
album[figu] = 1
```

- Importa `random`

En nuestro caso, nos alcanza con la función `random.randint(a,b)`, que nos permite generar un número al azar entre `a` y `b`. Por ejemplo:

```
import random

album = [0,0,0,0,0,0]
figu = random.randint(0,5)
album[figu] = 1
```

- Importa `random`
- crea el álbum, lleno de ceros (vacío)

En nuestro caso, nos alcanza con la función `random.randint(a,b)`, que nos permite generar un número al azar entre `a` y `b`. Por ejemplo:

```
import random

album = [0,0,0,0,0,0]
figu = random.randint(0,5)
album[figu] = 1
```

- Importa `random`
- crea el álbum, lleno de ceros (vacío)
- saca una figurita al azar entre las seis posibles (numerada de 0 a 5).

En nuestro caso, nos alcanza con la función `random.randint(a,b)`, que nos permite generar un número al azar entre `a` y `b`. Por ejemplo:

```
import random

album = [0,0,0,0,0,0]
figu = random.randint(0,5)
album[figu] = 1
```

- Importa `random`
- crea el álbum, lleno de ceros (vacío)
- saca una figurita al azar entre las seis posibles (numerada de 0 a 5).
- La *pega* en el álbum (poniendo un 1 en el casillero correspondiente).

¿Cómo hacemos para repetir?

Queremos repetir muchas veces las últimas dos líneas de nuestro código. Es decir, nos gustaría hacer algo así:

```
import random

album = [0,0,0,0,0,0]
mientras el álbum no esté lleno:
    figu = random.randint(0,5)
    album[figu] = 1
```

¿Cómo hacemos para repetir?

Queremos repetir muchas veces las últimas dos líneas de nuestro código. Es decir, nos gustaría hacer algo así:

```
import random

album = [0,0,0,0,0,0]
mientras el álbum no esté lleno:
    figu = random.randint(0,5)
    album[figu] = 1
```

Pero ¿cómo se programa ese *mientras*?

CICLOS CON WHILE

while

- Para “*Mientras*” usamos `while`, que es la traducción del inglés:
`while` *el álbum no esté lleno*:

while

- Para “*Mientras*” usamos `while`, que es la traducción del inglés:
`while` *el álbum no esté lleno*:
- Nos queda cómo indicar en `Python` que el álbum está lleno. ¿Alguna idea?

Álbum Ileno

Si un 0 indica la *ausencia* de figuritas y un 1 indica que la figurita está pegada. . .

Álbum lleno

Si un 0 indica la *ausencia* de figuritas y un 1 indica que la figurita está pegada. . .
... podemos **sumar** la lista `album`:

- Si la suma del álbum es **menor** que su longitud (6), entonces es porque aún quedan casilleros vacíos.
- Cuando la suma del álbum es **exactamente** la longitud (6), el álbum está lleno y queremos dejar de comprar figus.

Álbum lleno

Si un 0 indica la *ausencia* de figuritas y un 1 indica que la figurita está pegada. . .
... podemos **sumar** la lista `album`:

- Si la suma del álbum es **menor** que su longitud (6), entonces es porque aún quedan casilleros vacíos.
- Cuando la suma del álbum es **exactamente** la longitud (6), el álbum está lleno y queremos dejar de comprar figus.

De este modo, nuestro *ciclo* sería:

```
while sum(album) < 6:
```

Programa

Poniendo todo junto nuestro programa queda:

```
import random

album = [0,0,0,0,0,0]
while sum(album)<6:
    figu = random.randint(0,5)
    album[figu] = 1
```

Programa

Poniendo todo junto nuestro programa queda:

```
import random

album = [0,0,0,0,0,0]
while sum(album)<6:
    figu = random.randint(0,5)
    album[figu] = 1
```

Importante

El `while` **sabe** qué instrucciones se repiten usando la separación del borde (conocida como **tabulación**). Es algo específico de `Python`, otros lenguajes usan otras formas de indicar qué parte es la que se repite.

Programa

Poniendo todo junto nuestro programa queda:

```
import random

album = [0,0,0,0,0,0]
while sum(album)<6:
    figu = random.randint(0,5)
    album[figu] = 1
```

Importante

El `while` **sabe** qué instrucciones se repiten usando la separación del borde (conocida como **tabulación**). Es algo específico de `Python`, otros lenguajes usan otras formas de indicar qué parte es la que se repite.

Alerta posible metida de pata

Al final del `while` hay que poner los dos puntos (:). Si nos olvidamos, da error.

Cosas que faltan

Nuestro programa funciona, pero no nos dice mucho.

- No sabemos cuántas figuritas hubo que comprar para llenar el álbum.

Cosas que faltan

Nuestro programa funciona, pero no nos dice mucho.

- No sabemos cuántas figuritas hubo que comprar para llenar el álbum.
- En muchos lugares estamos suponiendo que el álbum tiene exactamente 6 casilleros para figuritas. Si queremos pasar al álbum de verdad, hay muchas correcciones que hacer.

¿Cuántas figuritas?

Para llevar la cuenta de cuántas figuritas compramos necesitamos introducir una **nueva variable** que se ocupe específicamente de eso.

¿Cuántas figuritas?

Para llevar la cuenta de cuántas figuritas compramos necesitamos introducir una **nueva variable** que se ocupe específicamente de eso.

Esta nueva variable:

- comienza valiendo 0 (al principio no compramos ninguna figu).

¿Cuántas figuritas?

Para llevar la cuenta de cuántas figuritas compramos necesitamos introducir una **nueva variable** que se ocupe específicamente de eso.

Esta nueva variable:

- comienza valiendo 0 (al principio no compramos ninguna figu).
- aumenta 1 cada vez que sacamos una figu nueva (repetida o no).

¿Cuántas figuritas?

Para llevar la cuenta de cuántas figuritas compramos necesitamos introducir una **nueva variable** que se ocupe específicamente de eso.

Esta nueva variable:

- comienza valiendo 0 (al principio no compramos ninguna figu).
- aumenta 1 cada vez que sacamos una figu nueva (repetida o no).

¿Cuántas figuritas?

Para llevar la cuenta de cuántas figuritas compramos necesitamos introducir una **nueva variable** que se ocupe específicamente de eso.

Esta nueva variable:

- comienza valiendo 0 (al principio no compramos ninguna figu).
- aumenta 1 cada vez que sacamos una figu nueva (repetida o no).

```
import random

album = [0,0,0,0,0,0]
contador_figus = 0
while sum(album)<6:
    figu = random.randint(0,5)
    album[figu] = 1
    contador_figus = contador_figus + 1
print(contador_figus)
```

Observaciones

- `contador_figus` es lo que habitualmente se llama un **contador**. Es una variable como cualquier otra, pero su función es *contar* algo.

Observaciones

- `contador_figus` es lo que habitualmente se llama un **contador**. Es una variable como cualquier otra, pero su función es *contar* algo.
- `contador_figus = contador_figus + 1`: esta línea es **super importante** porque muestra que la asignación **no** es lo mismo que una igualdad matemática: es una **operación**, que debe leerse de derecha a izquierda:

Observaciones

- `contador_figus` es lo que habitualmente se llama un **contador**. Es una variable como cualquier otra, pero su función es *contar* algo.
- `contador_figus = contador_figus + 1`: esta línea es **super importante** porque muestra que la asignación **no** es lo mismo que una igualdad matemática: es una **operación**, que debe leerse de derecha a izquierda:
 - Se lee el valor de `contador_figus` y se le suma 1.

Observaciones

- `contador_figus` es lo que habitualmente se llama un **contador**. Es una variable como cualquier otra, pero su función es *contar* algo.
- `contador_figus = contador_figus + 1`: esta línea es **super importante** porque muestra que la asignación **no** es lo mismo que una igualdad matemática: es una **operación**, que debe leerse de derecha a izquierda:
 - Se lee el valor de `contador_figus` y se le suma 1.
 - El resultado de esa operación se almacena en la variable `contador_figus`.

Observaciones

- `contador_figus` es lo que habitualmente se llama un **contador**. Es una variable como cualquier otra, pero su función es *contar* algo.
- `contador_figus = contador_figus + 1`: esta línea es **super importante** porque muestra que la asignación **no** es lo mismo que una igualdad matemática: es una **operación**, que debe leerse de derecha a izquierda:
 - Se lee el valor de `contador_figus` y se le suma 1.
 - El resultado de esa operación se almacena en la variable `contador_figus`.
 - En la práctica esto **sobreescribe** `contador_figus` con un nuevo valor (el anterior + 1).

Observaciones

- `contador_figus` es lo que habitualmente se llama un **contador**. Es una variable como cualquier otra, pero su función es *contar* algo.
- `contador_figus = contador_figus + 1`: esta línea es **super importante** porque muestra que la asignación **no** es lo mismo que una igualdad matemática: es una **operación**, que debe leerse de derecha a izquierda:
 - Se lee el valor de `contador_figus` y se le suma 1.
 - El resultado de esa operación se almacena en la variable `contador_figus`.
 - En la práctica esto **sobreescribe** `contador_figus` con un nuevo valor (el anterior + 1).
- `print(contador_figus)` está línea está **fuera del while**, porque no está tabulada. Por lo tanto, sólo se imprimirá el valor **una vez**, cuando el álbum ya esté lleno.

Cambiando el tamaño del álbum

El tamaño del álbum se presupone seis (6) en las siguientes líneas:

- `album=[0,0,0,0,0,0]`
- `while sum(album)<6:`
- `figu = random.randint(0,5)`

Cambiar esto es complicado y se presta a error. Sobre todo si nuestro programa sigue creciendo.

Cambiando el tamaño del álbum

El tamaño del álbum se presupone seis (6) en las siguientes líneas:

- `album=[0,0,0,0,0,0]`
- `while sum(album)<6:`
- `figu = random.randint(0,5)`

Cambiar esto es complicado y se presta a error. Sobre todo si nuestro programa sigue creciendo.

Nos conviene introducir una nueva variable que indique el tamaño del álbum y que se defina una única vez.

Cambiando el tamaño del álbum

El tamaño del álbum se presupone seis (6) en las siguientes líneas:

- `album=[0,0,0,0,0,0]`
- `while sum(album)<6:`
- `figu = random.randint(0,5)`

Cambiar esto es complicado y se presta a error. Sobre todo si nuestro programa sigue creciendo.

Nos conviene introducir una nueva variable que indique el tamaño del álbum y que se defina una única vez.

Las líneas anteriores deberían pasar a depender de esta nueva variable.

El código completo

```
import random

tam_album = 6
album = [0]*tam_album
contador_figus = 0
while sum(album)<tam_album:
    figu = random.randint(0,tam_album-1)
    album[figu] = 1
    contador_figus = contador_figus + 1
print(contador_figus)
```

El código completo

```
import random

tam_album = 6
album = [0]*tam_album
contador_figus = 0
while sum(album)<tam_album:
    figu = random.randint(0,tam_album-1)
    album[figu] = 1
    contador_figus = contador_figus + 1
print(contador_figus)
```

Esto recién comienza. . .

Terminamos. . . ¡sólo por hoy!